UM HACKATHON 2026

System Analysis Documentation

Team Free Riders

AI-Powered Food Waste Redistribution Platform

TEAM MEMBERS
1. JAYDEN FOO LOK HIN
2. VINCENT TEO KAI QI
3. ELVAN SEA MENG JIN
4. KWAN CHUN HOE
5. LIOW YEE SHUEN

Table of Contents

1. Introduction 3
   1.1 Purpose 3
      1.1.1 Architecture 3
      1.1.2 Data Flows 3
      1.1.2 Model Process 3
      1.1.4 Role of Reference 4
   1.2 Background 4
   1.3 Target Stakeholder 5
2. System Architecture & Design 6
   2.1 High Level Architecture 6
      2.1.1 Overview 6
      2.1.2 LLM as Service Layer 6
      2.1.3 Dependency Diagram 7
      2.1.4 Sequence Diagram 7
   2.2 Technological Stack 8
   2.3 Key Data Flows 8
      2.3.1 Data Flow Diagram 8
      2.3.2 Normalized Database Schema 9
3. Functional Requirements & Scopes 10
   3.1 Minimum Viable Product 10
   3.2 Non-Functional Requirements (NFRs) 10
   3.3 Out of Scope / Future Enhancements 11
4. Monitor, Evaluation, Assumptions & Dependencies 12
   4.1 Technical Evaluation: 12
   4.2 Monitoring 13
   4.3 Assumptions 13
   4.4 External Dependencies 14
5. Project Management & Team Contributions 16
   5.1 Project Timeline 16
   5.2 Team Members Role 16
   5.3 Recommendations 17

1. Introduction

1.1 Purpose

The System Analysis Documentation (SAD) here provides a comprehensive blueprint for our AI-Powered Food Waste Redistribution Platform which connects restaurants with surplus foods to Non-Governmental Organizations (NGOs) in Malaysia.

1.1.1 Architecture

The system is designed to be a cloud-based platform with microservices. The core components of the system such as the API gateway (Handler), response processing (Orchestrator), front-end services and integration are decoupled and operate separately. This allows for scalability, fault isolation and easier maintenance. The platform is suitable to be deployed on cloud infrastructure such as Azure or AWS, enabling high availability and real-time processing

1.1.2 Data Flows

The system handles data flow in a structured pipeline:

1. Restaurants submit donation details via a user interface using natural language input.
2. The input is sent to the Z.AI LLM service, which converts it into structured JSON data.
3. The structured data is processed by the matching engine to identify suitable NGOs.
4. A donation request is sent to selected NGOs through the notification system.
5. NGOs respond (accept/reject), and the response is recorded in the system.
6. All data is stored in a centralized database for persistence, tracking, and analytics.
7. Status updates are communicated back to the restaurant in real time.

1.1.2 Model Process

The model represents the end-to-end workflow of the core system functionality:

1. Submission of donation request by restaurant
2. AI-based data extraction and structuring
3. Intelligent NGO matching and ranking
4. Notification and request dispatch to NGOs

5. NGO decision (accept/reject)
6. Confirmation and coordination of food pickup
7. Completion and logging of donation activity

This workflow ensures minimal manual intervention while maintaining flexibility and transparency.

1.1.4 Role of Reference

This document serves primarily as a reference for developers, system architects. Stakeholders and testers. It provides a clear outline of the systems's architecture, data handling and workflow processes, ensuring consistency in implementation, development and testing. It also acts as a guide for stakeholders to understand the technical foundation and operational capabilities of the platform.

1.2 Background

As of 2025, Malaysia generates approximately 17,000 tonnes of daily food waste, with 24% of them being edible. Simultaneously, over 200 NGOs and community kitchens struggle with inconsistent food supplies due to fragmented coordination and lack of donators. The existing workflow of collaboration relies on:

- WhatsApp messages/Phone calls between restaurants and NGOs
- Manual spreadsheet tracking of donations
- Ad-hoc pickup arrangements without optimization
- No visibility of food availability amongst organizations

As there is no prior system to our proposed platform, these are the limitations to the current non-automated, manual process:

Current Method	Limitation
WhatsApp groups	Message flooding, no structured data & prioritization
Phone calls	Time-consuming, NGO availability unknown
Direct drop-off	Time-consuming, inefficient routing
Spreadsheet tracking of donations	Manual data entry, risk of data loss

These are the changes which are introduced in the proposed system:

1. AI-Powered Extraction – the GLM converts unstructured user prompts and messages into structured data, making it easier for messages to be deciphered by NGOs
2. Intelligent NGO ranking – Automatically matches a restaurant to a NGO
3. Stateful Workflow Engine – Tracks donation lifecycle from listing to completion
4. Real-Time Coordination – Instant notifications to notify NGOs
5. Transparent AI Reasoning – Decision logging for accountability

1.3 Target Stakeholder

Below are the stakeholders involved in our system:

Stakeholder	Role	Expectations
Restaurant Staff	Donate surplus food by submitting donation listings	Quick submission process; no complex forms
NGO Coordinator	View, prioritize and claim suitable food listings	Real-time visibility of food, clear pickup instructions
Development Team	Builds & maintain the system	Clear API calls, well-documented code
QA Team	Validates system behavior	Defines test scopes, creates test plans
System Administrator	Monitor platform health and traffic	Monitors usage analytics, failure alerts
Pickup Volunteer	Collects food from restaurants	Optimized routes, clear pickup windows

2. System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview

Type	Details
System	Web application (System Portal) + mobile/web interface
Architecture	Cloud-based microservices architecture with AI service layering

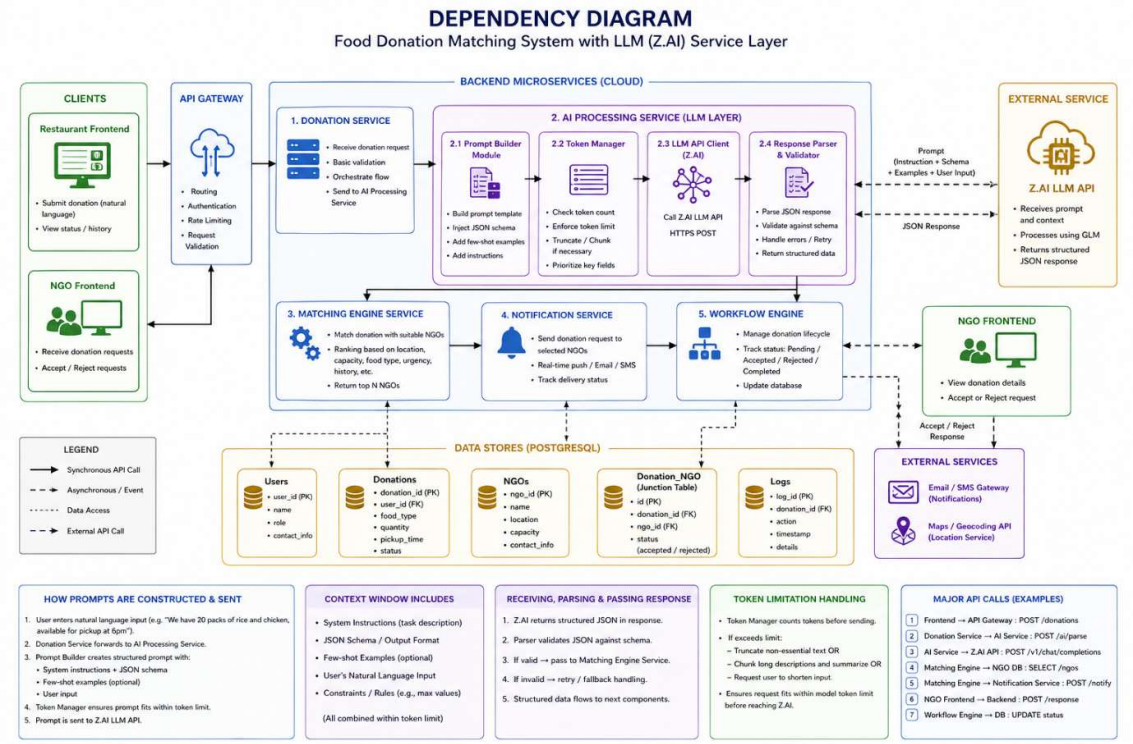
2.1.2 LLM as Service Layer

GLM is integrated as the central reasoning engine – not a generic "AI module." The architecture ensures that removing GLM breaks the coordination workflow.

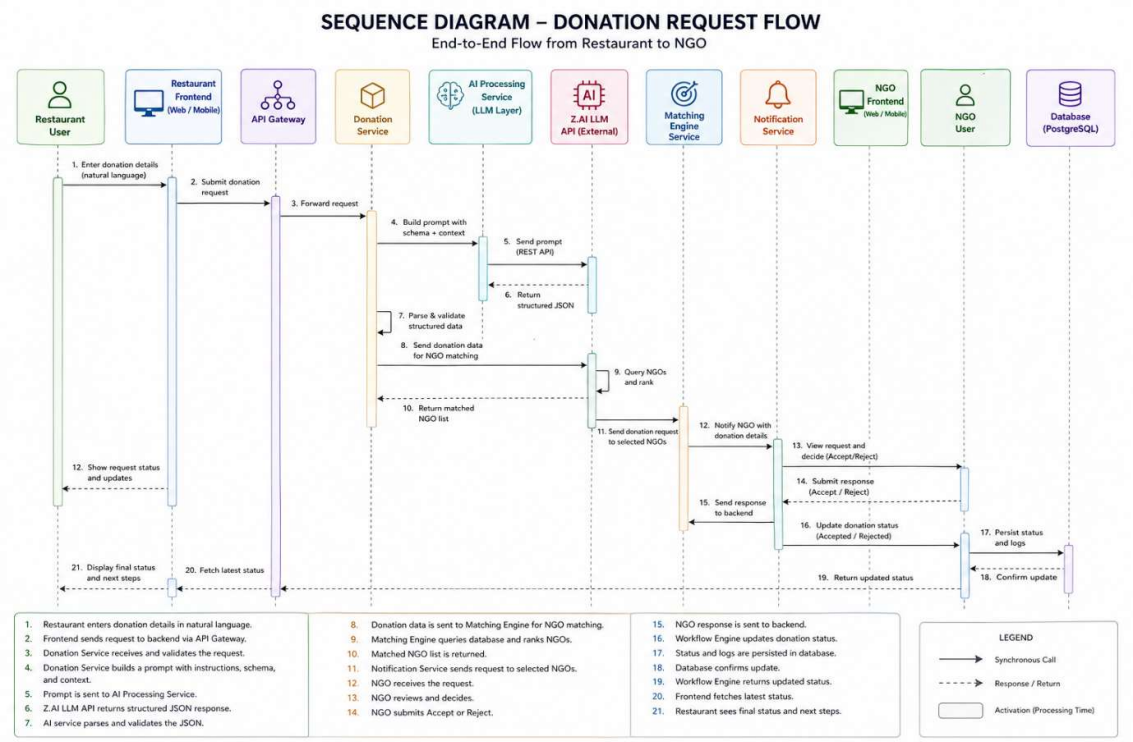
Integration Depth:

Layer	GLM Responsibility	Without GLM
Input Processing	Parse unstructured donation messages	Cannot extract structured data
NGO Ranking	Multi-factor suitability scoring	Falls back to distance-only (ineffective)
Clarification	Generate context-aware questions	No intelligent follow-up
Validation	Detect ambiguous/unsafe inputs	Rule-based misses edge cases
Decision Logging	Provide reasoning for each ranking	No transparency

2.1.3 Dependency Diagram



2.1.4 Sequence Diagram

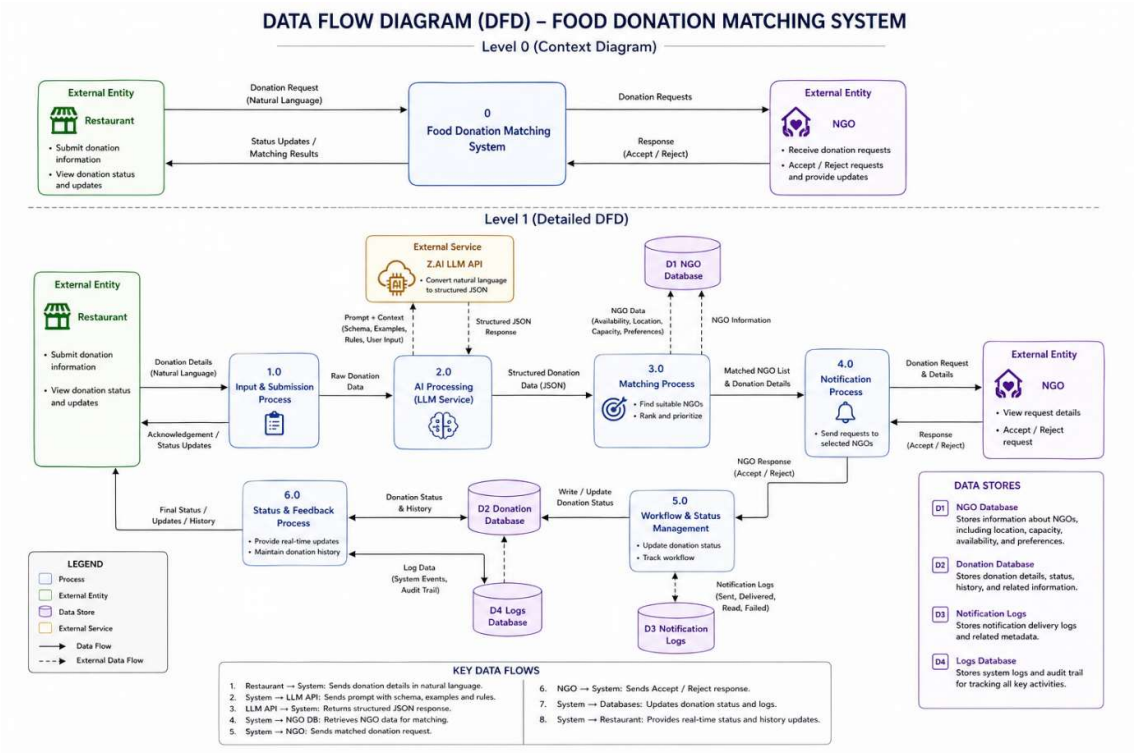


2.2 Technological Stack

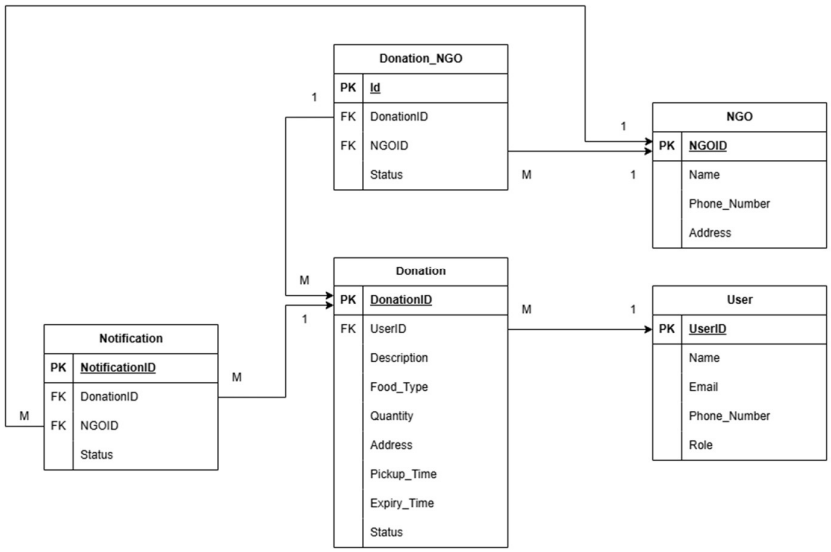
1. Frontend HTML / CSS / JavaScript (cross-platform support for web and mobile users)
2. Backend: Z.AI LLM & ILMU LLM API calls
3. Database: PostgreSQL (relational database for structured data storage)
4. Cloud/Deployment:
    - AWS EC2 / GCP Compute Engine (application hosting)
    - AWS RDS (PostgreSQL database)
    - AWS S3 (static assets storage)
    - Optional: Redis (caching), RabbitMQ/Kafka (message queues)

2.3 Key Data Flows

2.3.1 Data Flow Diagram



2.3.2 Normalized Database Schema



This schema satisfies **Third Normal Form (3NF)**:

1. First Normal Form (1NF)

- All fields are atomic
- Each table has a primary key

2. Second Normal Form (2NF)

- No partial dependencies
- All non-key attributes fully depend on the primary key

3. Third Normal Form (3NF)

- No transitive dependencies
- Example:
    - NGO details are stored only in NGO and not duplicated in DONATION
    - User info is separated into USER
    - Matching relationship handled via DONATION_NGO

3. Functional Requirements & Scopes

3.1 Minimum Viable Product

Feature	Description	AI Dependency
Unstructured Donation Input	Restaurant staff types natural language message; GLM extracts structured data (restaurant name, food items, quantity, time, contact)	Core GLM
Intelligent NGO Ranking	GLM ranks NGOs based on food type compatibility, capacity, distance, urgency, operating hours	Core GLM
Stateful Workflow Tracking	Donation moves through states: LISTED → AI_RANKING → NOTIFIED → CLAIMED → PICKUP_SCHEDULED → COMPLETED	System logic
Clarification Handling	Missing/ambiguous fields trigger AI-generated follow-up questions with conversation context	Core GLM
Pickup Route Visualization	Display restaurant and NGO locations on Google Maps with optimized route	External API

3.2 Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Scalability	System must handle 50 concurrent donation submissions during peak hours	Docker container orchestration with Kubernetes; PostgreSQL connection pooling (SQLAlchemy with pool size 20-50); horizontal scaling with load balancer; read replicas for database queries
Reliability	Donation submission success rate > 99% under normal load	Retry logic for GLM calls (max 3 retries)
Availability	Core functionality available 99.5% during hackathon demo	Firebase SLA 99.95%; graceful degradation if GLM unavailable (return error, not fallback)
Latency (GLM)	GLM API response within 5 seconds at p95	30-second timeout; async processing; user-facing loading state
Token Efficiency	Average token cost <1,200 per donation session	Prompt truncation at 2000 chars; cache NGO profiles; limit ranking to top 5

Maintainability	Modular separation between LLM handler, tools, and orchestrator	Clear interface boundaries; dependency injection ready
Security	Restaurant and NGO authentication; data isolation	JWT-based authentication; PostgreSQL row-level security (RLS); environment variables for secrets (never hardcoded); Docker secrets management; HTTPS enforcement; input validation and sanitization; rate limiting per user/session
Observability	AI decisions logged with reasoning for audit	Decision log collection; confidence score tracking

3.3 Out of Scope / Future Enhancements

Feature	Reason for Exclusion	Target Phase
Mobile native apps	Web PWA sufficient for MVP	Phase 2
Real-time GPS tracking	Complexity exceeds hackathon scope	Phase 2
QR code pickup verification	Nice-to-have; manual confirmation works	Phase 2
NGO rating/reputation system	Requires extensive user base	Phase 3
Recurring donation scheduling	Beyond core MVP	Phase 3
Multi-language UI (Chinese, Tamil)	English/Malay sufficient for demo	Phase 3
Analytics dashboard	Basic logging only for MVP	Phase 2
Automated WhatsApp integration	API complexity; manual entry works	Phase 3
Donation tax receipt generation	Legal compliance complexity	Phase 4
Blockchain verification	Over-engineering for problem scope	Not planned

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation:

Grayscale Rollout Strategy

Although FoodLoop is a hackathon project, the following strategy would apply for production:

Phase	User Percentage	Duration	Success Criteria
Phase 0	Internal testers (5 restaurants, 5 NGOs)	2 days	Zero critical bugs
Phase 1	5% of registered restaurants	1 day	GLM response <5s; no rejected valid donations
Phase 2	25% of users	2 days	NGO claim rate >70%; no hallucinated rankings
Phase 3	100% rollout	-	All metrics stable >24 hours

A/B Testing (Post-MVP)

- **Variant A:** Current ranking algorithm (GLM only)
- **Variant B:** GLM + rule-based fallback for low-confidence
- **Metric:** NGO claim rate within 10 minutes

Emergency Rollback (ER) & Golden Release

Trigger Condition	Action	Recovery Time
GLM error rate >10% in 5 minutes	Return graceful error; display maintenance message	Rolling back to previous stable version
Hallucination detection (food type mismatch)	manual override; admin notification	<15 minutes
Data corruption in Firestore	Restore from backup (hourly snapshots)	<30 minutes

Golden Release Artifacts:

- Stable commit hash tagged v1.0.0-golden
- Environment variables documented
- Rollback script tested

4.2 Monitoring

Key Metrics Dashboard:

Metric	Threshold	Alert Action
GLM API latency (p95)	>5 seconds	Warning; investigate prompt size
GLM error rate	>5%	Critical; emergency rollback
Donation success rate	<95%	High priority; check orchestrator
NGO claim rate within 15 min	<60%	Medium; review ranking quality
Hallucinated responses	>2%	High; review prompt engineering
Firestore write failures	>1%	Critical; check quota

Logging Strategy:

Log Type	Destination	Retention
GLM requests/responses	Cloud Logging	7 days
AI decision reasoning	Firestore (decisions collection)	30 days
User actions (donate, claim)	Analytics	90 days
Error stack traces	Sentry (if integrated)	90 days

4.3 Assumptions

Assumption	Validation Method
Restaurant staff have basic smartphone/web access	Target demographic verified
NGOs have staff to monitor dashboard during operating hours	NGO partner agreement
Users can type/voice basic English or Malay	UI provides examples in both languages

Pickup volunteers have personal vehicles	NGO responsibility
GLM API remains available during hackathon demo	Mock mode prepared as backup
Google Maps API quota sufficient for demo (<28k requests)	Conservative usage estimated
Users understand food safety basics (no expired donations)	System validates and rejects
Stable internet connection for all users	Assumed for demo environment

4.4 External Dependencies

Dependency	Purpose	Risk Level	Mitigation Strategy
Z.AI GLM API	Core LLM reasoning engine for extraction, ranking, clarification	HIGH	Mock LLM handler for demo; graceful error states; no rule-based fallback (proves necessity)
Google Maps API	Geocoding addresses; route visualization	MEDIUM	Fallback to text-based distance estimation; cache geocoding results
PostgreSQL	Persistent database storage for NGOs, donations, claims, and session data	MEDIUM	Daily automated backups; connection pooling; read replicas for scalability; fallback to in-memory cache if database unreachable
Docker	Containerization for a consistent development and deployment environment	LOW	Multi-stage builds for optimization; Docker Compose for local development; container health checks; automatic restart on failure

HTML/CSS	Frontend user interface for restaurants and NGOs dashboards	LOW	Responsive design for mobile/desktop; progressive enhancement; fallback to basic HTML if CSS fails

5. Project Management & Team Contributions

5.1 Project Timeline

Day	Phase	Activities	Deliverables
1-2	Planning & Setup	Environment setup; Firebase project; Git repo; API keys	Working dev environment
3-4	Core AI Integration	LLM handler implementation; prompt engineering; GLM integration	GLM extracting structured data
5-6	Orchestrator & Workflow	State machine; conversation history; routing logic	Full orchestrator working
7-8	Frontend & Integration	React dashboard; NGO listing view; map integration	End-to-end flow functional
9	Testing & Polish	Edge cases; error handling; demo script	All test cases passing
10	Documentation & Submission	PRD, SAD, demo recording, pitch deck	Complete submission

5.2 Team Members Role

Role	Responsibilities	Key Deliverables
AI/LLM Lead	Prompt engineering; GLM integration; response parsing; hallucination mitigation	LLM handler; NGO ranking logic
Backend Lead	Orchestrator; FastAPI; state machine; tool integration (geocoding, time parser)	Orchestrator; API endpoints; Firestore schema
Frontend Lead	React dashboard; NGO/restaurant views; map integration; real-time updates	Web interface; Google Maps integration
QA & Testing	Test cases; edge scenario validation; demo preparation	Bug reports; edge case coverage; demo script
Documentation	PRD, SAD, technical documentation	Complete documentation package

5.3 Recommendations

For Scalability (Post-Hackathon):

Recommendation	Justification	Effort
Implement Redis caching	Cache NGO profiles and frequent geocoding results; reduce GLM token cost by ~30%	Medium
Add async task queue (Celery)	Handle batch ranking for multiple donations; prevent timeout on concurrent requests	High
Implement rate limiting	Prevent abuse; manage GLM API quota	Low

For Performance (Immediate):

Recommendation	Justification	Effort
Prompt compression	Reduce token count by 20% via concise examples	Low
NGO profile partial updates	Only changed fields submitted to GLM	Low
Lazy loading of maps	Improve initial load time	Low

For Reliability (Critical):

Recommendation	Justification	Effort
Idempotent donation creation	Prevent duplicate listings on retry	Medium
Transaction for claim process	Prevent double-claiming race conditions	Medium
Dead letter queue for failed GLM calls	Audit and retry failed requests	High (post-MVP)